Red Hat AI Inference 3.4

Deploy the standalone Red Hat AI Inference container in OpenShift Container Platform

Deploy the standalone Red Hat AI Inference container in OpenShift Container Platform clusters that have supported AI accelerators installed

Last Updated: 2026-05-21

Red Hat AI Inference 3.4 Deploy the standalone Red Hat AI Inference container in OpenShift Container Platform

Deploy the standalone Red Hat AI Inference container in OpenShift Container Platform clusters that have supported AI accelerators installed

Legal Notice

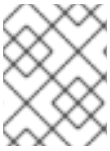
Abstract

Learn how to work with Red Hat AI Inference for model serving and inferencing.

Table of Contents

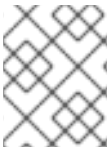
CHAPTER 1. ABOUT STANDALONE OPENSHIFT CONTAINER PLATFORM DEPLOYMENTS

You can deploy the standalone Red Hat AI Inference container in OpenShift Container Platform clusters with supported AI accelerators that have full access to the internet. A standalone deployment runs a single AI Inference container instance directly as an OpenShift Container Platform **Deployment** resource.



NOTE

To deploy models at scale with intelligent request scheduling, KV cache management, and advanced deployment patterns, use Distributed Inference with llm-d instead.



NOTE

Install the NVIDIA GPU Operator or AMD GPU Operator as appropriate for the underlying host AI accelerators that are available in the cluster.

Deploying the standalone Red Hat AI Inference container in OpenShift Container Platform requires installing the Node Feature Discovery (NFD) Operator to detect hardware capabilities, then installing the appropriate GPU operator for your accelerator hardware. After the operators are configured, you can deploy inference workloads using Red Hat AI Inference container images.

CHAPTER 2. INSTALLING THE NODE FEATURE DISCOVERY OPERATOR

Install the Node Feature Discovery Operator so that the cluster can use the AI accelerators that are available in the cluster.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.

Procedure

1. Create the **Namespace** CR for the Node Feature Discovery Operator:

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-nfd
  labels:
    name: openshift-nfd
    openshift.io/cluster-monitoring: "true"
EOF
```

2. Create the **OperatorGroup** CR:

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  generateName: openshift-nfd-
  name: openshift-nfd
  namespace: openshift-nfd
spec:
  targetNamespaces:
  - openshift-nfd
EOF
```

3. Create the **Subscription** CR:

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: nfd
  namespace: openshift-nfd
spec:
  channel: "stable"
  installPlanApproval: Automatic
  name: nfd
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

```
EOF
```

Verification

Verify that the Node Feature Discovery Operator deployment is successful by running the following command:

```
$ oc get pods -n openshift-nfd
```

Example output

```
NAME                                  READY   STATUS    RESTARTS   AGE
nfd-controller-manager-7f86ccfb58-vgr4x   2/2     Running   0          10m
```

Additional resources

- Installing the Node Feature Discovery Operator

CHAPTER 3. INSTALLING THE NVIDIA GPU OPERATOR

Install the NVIDIA GPU Operator to use the underlying NVIDIA CUDA AI accelerators that are available in the cluster.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have installed the Node Feature Discovery Operator.

Procedure

1. Create the **Namespace** CR for the NVIDIA GPU Operator:

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: nvidia-gpu-operator
EOF
```

2. Create the **OperatorGroup** CR:

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: gpu-operator-certified
  namespace: nvidia-gpu-operator
spec:
 targetNamespaces:
 - nvidia-gpu-operator
EOF
```

3. Create the **Subscription** CR:

```
oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: gpu-operator-certified
  namespace: nvidia-gpu-operator
spec:
  channel: "stable"
  installPlanApproval: Manual
  name: gpu-operator-certified
  source: certified-operators
  sourceNamespace: openshift-marketplace
EOF
```

Verification

Verify that the NVIDIA GPU Operator deployment is successful by running the following command:

```
$ oc get pods -n nvidia-gpu-operator
```

Example output

```
NAME                                       READY   STATUS      RESTARTS   AGE
gpu-feature-discovery-c2rfm                1/1     Running     0          6m28s
gpu-operator-84b7f5bcb9-vqds7              1/1     Running     0          39m
nvidia-container-toolkit-daemonset-pgcrf   1/1     Running     0          6m28s
nvidia-cuda-validator-p8gv2                0/1     Completed   0          99s
nvidia-dcgm-exporter-kv6k8                 1/1     Running     0          6m28s
nvidia-dcgm-tpsps                          1/1     Running     0          6m28s
nvidia-device-plugin-daemonset-gbn55       1/1     Running     0          6m28s
nvidia-device-plugin-validator-z7ltr       0/1     Completed   0          82s
nvidia-driver-daemonset-410.84.202203290245-0-xxgdv   2/2     Running     0          6m28s
nvidia-node-status-exporter-snmsm          1/1     Running     0          6m28s
nvidia-operator-validator-6pfk6            1/1     Running     0          6m28s
```

Additional resources

- Installing the NVIDIA GPU Operator

CHAPTER 4. INSTALLING THE AMD GPU OPERATOR

Install the AMD GPU Operator to use the underlying AMD ROCm AI accelerators that are available in the cluster.

Installing the AMD GPU Operator is a multi-step procedure that requires installing the Node Feature Discovery Operator, the Kernel Module Management Operator (KMM), and then the AMD GPU Operator through the OpenShift OperatorHub.

IMPORTANT

The AMD GPU Operator is only supported in clusters with full access to the internet, not in disconnected environments. This is because the Operator builds the driver inside the cluster which requires full internet access.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have installed the following Operators in the cluster:

  Table 4.1. Required Operators

Operator	Description
Service CA Operator	Issues TLS serving certificates for Service objects. Required for certificate signing and authentication between the **kube-apiserver** and the KMM webhook server.
Operator Lifecycle Manager (OLM)	Manages Operator installation and lifecycle maintenance.
Machine Config Operator	Manages the operating system configuration of worker and control-plane nodes. Required for configuring the kernel blacklist for the amdgpu driver.
Cluster Image Registry Operator	The Cluster Image Registry Operator (CIRO) manages the internal container image registry that OpenShift Container Platform clusters use to store and serve container images. Required for driver image building and storage in the cluster.

Procedure

1. Create the **Namespace** CR for the AMD GPU Operator Operator:

```
oc apply -f - <<EOF
apiVersion: v1
kind: Namespace
metadata:
  name: openshift-amd-gpu
  labels:
```

```
    name: openshift-amd-gpu
    openshift.io/cluster-monitoring: "true"
EOF
```

2. Verify that the Service CA Operator is operational. Run the following command:

```
$ oc get pods -A | grep service-ca
```

Example output

```
openshift-service-ca-operator   service-ca-operator-7cfd997ddf-llhdg    1/1    Running    7
35d
openshift-service-ca            service-ca-8675b766d5-vz8gg             1/1    Running    6    35d
```

3. Verify that the Machine Config Operator is operational:

```
$ oc get pods -A | grep machine-config-daemon
```

Example output

```
openshift-machine-config-operator   machine-config-daemon-sdsjj   2/2    Running    10   35d
openshift-machine-config-operator   machine-config-daemon-xc6rm   2/2    Running    0
2d21h
```

4. Verify that the Cluster Image Registry Operator is operational:

```
$ oc get pods -n openshift-image-registry
```

Example output

```
NAME                                               READY   STATUS      RESTARTS   AGE
cluster-image-registry-operator-58f9dc9976-czt2w   1/1     Running     5          35d
image-pruner-29259360-2tdrk                        0/1     Completed   0          2d8h
image-pruner-29260800-v9lkc                        0/1     Completed   0          32h
image-pruner-29262240-swcmb                        0/1     Completed   0          8h
image-registry-7b67584cd-sdxpk                     1/1     Running     10         35d
node-ca-d2kzl                                      1/1     Running     0          2d21h
node-ca-xxzrw                                      1/1     Running     5          35d
```

5. Optional: If you plan to build driver images in the cluster, you must enable the OpenShift internal registry. Run the following commands:

   a. Verify current registry status:

```
$ oc get pods -n openshift-image-registry
```

```
NAME                                    READY   STATUS      RESTARTS   AGE
#...
image-registry-7b67584cd-sdxpk          1/1     Running     10         36d
```

   b. Configure the registry storage. The following example patches an **emptyDir** ephemeral volume in the cluster. Run the following command:

```
$ oc patch configs.imageregistry.operator.openshift.io cluster --type merge \
  --patch '{"spec":{"storage":{"emptyDir":{}}}}'
```

c. Enable the registry:

```
$ oc patch configs.imageregistry.operator.openshift.io cluster --type merge \
  --patch '{"spec":{"managementState":"Managed"}}'
```

6. Install the Node Feature Discovery (NFD) Operator. See Installing the Node Feature Discovery Operator.

7. Install the Kernel Module Management (KMM) Operator. See Installing the Kernel Module Management Operator.

8. Configure node feature discovery for the AMD AI accelerator:

a. Create a **NodeFeatureDiscovery** (**NFD**) custom resource (CR) to detect AMD GPU hardware. For example:

```
apiVersion: nfd.openshift.io/v1
kind: NodeFeatureDiscovery
metadata:
  name: amd-gpu-operator-nfd-instance
  namespace: openshift-nfd
spec:
  workerConfig:
    configData: |
      core:
        sleepInterval: 60s
      sources:
        pci:
          deviceClassWhitelist:
            - "0200"
            - "03"
            - "12"
          deviceLabelFields:
            - "vendor"
            - "device"
        custom:
        - name: amd-gpu
          labels:
            feature.node.kubernetes.io/amd-gpu: "true"
          matchAny:
            - matchFeatures:
                - feature: pci.device
                  matchExpressions:
                    vendor: {op: In, value: ["1002"]}
                    device: {op: In, value: [
                      "740f", # MI210
                    ]}
        - name: amd-vgpu
          labels:
            feature.node.kubernetes.io/amd-vgpu: "true"
          matchAny:
            - matchFeatures:
```

```
    - feature: pci.device
      matchExpressions:
        vendor: {op: In, value: ["1002"]}
        device: {op: In, value: [
          "74b5", # MI300X VF
        ]}
```

NOTE

Depending on your specific cluster deployment, you might require a **NodeFeatureDiscovery** or **NodeFeatureRule** CR. For example, the cluster might already have the **NodeFeatureDiscovery** resource deployed and you don't want to change it. For more information, see Create Node Feature Discovery Rule.

9. Create a **MachineConfig** CR to add the out-of-tree **amdgpu** kernel module to the modprobe blacklist. For example:

```
apiVersion: machineconfiguration.openshift.io/v1
kind: MachineConfig
metadata:
  labels:
    machineconfiguration.openshift.io/role: worker
  name: amdgpu-module-blacklist
spec:
  config:
    ignition:
      version: 3.2.0
    storage:
      files:
        - path: "/etc/modprobe.d/amdgpu-blacklist.conf"
          mode: 420
          overwrite: true
          contents:
            source: "data:text/plain;base64,YmxhY2tsaXN0IGFtZGdwdQo="
```

Where:

machineconfiguration.openshift.io/role: worker

Specifies the node role for the machine configuration. Set this value to **master** for single-node OpenShift clusters.

IMPORTANT

The Machine Config Operator automatically reboots selected nodes after you apply the **MachineConfig** CR.

10. Create the **DeviceConfig** CR to start the AMD AI accelerator driver installation. For example:

```
apiVersion: amd.com/v1alpha1
kind: DeviceConfig
metadata:
  name: driver-cr
```

```
  namespace: openshift-amd-gpu
spec:
  driver:
    enable: true
    image: image-registry.openshift-image-
registry.svc:5000/$MOD_NAMESPACE/amdgpu_kmod
    version: 6.2.2
  selector:
    "feature.node.kubernetes.io/amd-gpu": "true"
```

Where:

image: image-registry.openshift-image-registry.svc:5000/$MOD_NAMESPACE/amdgpu_kmod

Specifies the driver image location. By default, you do not need to configure a value for this field because the default value is used.

After you apply the **DeviceConfig** CR, the AMD GPU Operator collects the worker node system specifications, builds or retrieve the appropriate driver image, uses KMM to deploy the driver, and finally deploys the ROCM device plugin and node labeller.

Verification

1. Verify that the KMM worker pods are running:

```
$ oc get pods -n openshift-kmm
```

Example output

```
NAME                                  READY   STATUS    RESTARTS        AGE
kmm-operator-controller-774c7ccff6-hr76v   1/1     Running   30 (2d23h ago)   35d
kmm-operator-webhook-76d7b9555-ltmps       1/1     Running   5                35d
```

2. Check device plugin and labeller status:

```
$ oc -n openshift-amd-gpu get pods
```

Example output

```
NAME                                          READY   STATUS    RESTARTS       AGE
amd-gpu-operator-controller-manager-59dd964777-zw4bg   1/1     Running   8 (2d23h ago)
9d
test-deviceconfig-device-plugin-kbrp7                  1/1     Running   0              2d
test-deviceconfig-metrics-exporter-k5v4x               1/1     Running   0              2d
test-deviceconfig-node-labeller-fqz7x                  1/1     Running   0              2d
```

3. Confirm that GPU resource labels are applied to the nodes:

```
$ oc get node -o json | grep amd.com
```

Example output

```
"amd.com/gpu.cu-count": "304",
```

```
"amd.com/gpu.device-id": "74b5",
"amd.com/gpu.driver-version": "6.12.12",
"amd.com/gpu.family": "AI",
"amd.com/gpu.simd-count": "1216",
"amd.com/gpu.vram": "191G",
"beta.amd.com/gpu.cu-count": "304",
"beta.amd.com/gpu.cu-count.304": "8",
"beta.amd.com/gpu.device-id": "74b5",
"beta.amd.com/gpu.device-id.74b5": "8",
"beta.amd.com/gpu.family": "AI",
"beta.amd.com/gpu.family.AI": "8",
"beta.amd.com/gpu.simd-count": "1216",
"beta.amd.com/gpu.simd-count.1216": "8",
"beta.amd.com/gpu.vram": "191G",
"beta.amd.com/gpu.vram.191G": "8",
"amd.com/gpu": "8",
"amd.com/gpu": "8",
```


Additional resources

- Installing the Node Feature Discovery Operator
- Installing the Kernel Module Management Operator
- Installing the AMD GPU Operator

CHAPTER 5. DEPLOYING THE STANDALONE RED HAT AI INFERENCE CONTAINER AND INFERENCE SERVING THE MODEL

Deploy a language model with OpenShift Container Platform by configuring secrets, persistent storage, and a standalone **Deployment** custom resource (CR) that pulls the model from Hugging Face and uses Red Hat AI Inference to inference serve the model.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- You have installed NFD and the required GPU Operator for your underlying AI accelerator hardware.

Procedure

1. Create the **Secret** custom resource (CR) for the Hugging Face token. The cluster uses the **Secret** CR to pull models from Hugging Face.
   a. Set the **HF_TOKEN** variable using the token you set in Hugging Face.

   ```
   $ HF_TOKEN=<your_huggingface_token>
   ```

   b. Set the cluster namespace to match where you deployed the Red Hat AI Inference image, for example:

   ```
   $ NAMESPACE=rhaii-namespace
   ```

   c. Create the **Secret** CR in the cluster:

   ```
   $ oc create secret generic hf-secret --from-literal=HF_TOKEN=$HF_TOKEN -n
   $NAMESPACE
   ```

2. Create the Docker secret so that the cluster can download the Red Hat AI Inference image from the container registry. For example, to create a **Secret** CR that contains the contents of your local **~/.docker/config.json** file, run the following command:

   ```
   oc create secret generic docker-secret --from-file=.dockercfg=$HOME/.docker/config.json --
   type=kubernetes.io/dockercfg -n rhaii-namespace
   ```

3. Create a **PersistentVolumeClaim** (**PVC**) custom resource (CR) and apply it in the cluster. The following example **PVC** CR uses a default IBM VPC Block persistence volume. You use the **PVC** as the location where you store the models that you download.

   ```
   apiVersion: v1
   kind: PersistentVolumeClaim
   metadata:
     name: model-cache
     namespace: rhaii-namespace
   ```

```
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  storageClassName: ibmc-vpc-block-10iops-tier
```

NOTE

Configuring cluster storage to meet your requirements is outside the scope of this procedure. For more detailed information, see Configuring persistent storage.

4. Create a **Deployment** custom resource (CR) that pulls the model from Hugging Face and deploys the Red Hat AI Inference container. Reference the following example **Deployment** CR, which uses AI Inference to serve a Granite model on a CUDA accelerator.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: granite
  namespace: rhaii-namespace
  labels:
    app: granite
spec:
  replicas: 1
  selector:
    matchLabels:
      app: granite
  template:
    metadata:
      labels:
        app: granite
    spec:
      imagePullSecrets:
        - name: docker-secret
      volumes:
        - name: model-volume
          persistentVolumeClaim:
            claimName: model-cache
        - name: shm
          emptyDir:
            medium: Memory
            sizeLimit: "2Gi"
        - name: oci-auth
          secret:
            secretName: docker-secret
            items:
              - key: .dockercfg
                path: config.json
      serviceAccountName: default
      initContainers:
        - name: fetch-model
          image: ghcr.io/oras-project/oras:v1.2.0
```

```
        env:
          - name: DOCKER_CONFIG
            value: /auth
        command: ["/bin/sh","-c"]
        args:
          - |
            set -e
            # Only pull if /model is empty
            if [ -z "$(ls -A /model)" ]; then
              echo "Pulling model..."
              oras pull registry.redhat.io/rhelai1/granite-3-1-8b-instruct-quantized-w8a8:1.5 \
                --output /model \
            else
              echo "Model already present, skipping model pull"
            fi
        volumeMounts:
          - name: model-volume
            mountPath: /model
          - name: oci-auth
            mountPath: /auth
            readOnly: true
      containers:
        - name: granite
          image: 'registry.redhat.io/{rhaii-registry-namespace}/vllm-cuda-
rhel9@sha256:a6645a8e8d7928dce59542c362caf11eca94bb1b427390e78f0f8a87912041cd'

          imagePullPolicy: IfNotPresent
          env:
            - name: VLLM_SERVER_DEV_MODE
              value: '1'
          command:
            - python
            - '-m'
            - vllm.entrypoints.openai.api_server
          args:
            - '--port=8000'
            - '--model=/model'
            - '--served-model-name=granite-3-1-8b-instruct-quantized-w8a8'
            - '--tensor-parallel-size=1'
          resources:
            limits:
              cpu: '10'
              nvidia.com/gpu: '1'
              memory: 16Gi
            requests:
              cpu: '2'
              memory: 6Gi
              nvidia.com/gpu: '1'
          volumeMounts:
            - name: model-volume
              mountPath: /model
            - name: shm
              mountPath: /dev/shm
      restartPolicy: Always
```

+

Where:

namespace: rhaii-namespace

Specifies the deployment namespace. The value of **metadata.namespace** must match the namespace where you configured the Hugging Face **Secret** CR.

claimName: model-cache

Specifies the persistent volume claim name. The value of **spec.template.spec.volumes.persistentVolumeClaim.claimName** must match the name of the **PVC** that you created.

initContainers:

Defines a container that runs before the main application container to download the required model from Hugging Face. The model pull step is skipped if the model directory has already been populated, for example, from a previous deployment.

mountPath: /dev/shm

Mounts the shared memory volume required by the NVIDIA Collective Communications Library (NCCL). Tensor parallel vLLM deployments fail without this volume mount.

1. Increase the deployment replica count to the required number. For example, run the following command:

```
oc scale deployment granite -n rhaii-namespace --replicas=1
```

2. Optional: Watch the deployment and ensure that it succeeds:

```
$ oc get deployment -n rhaii-namespace --watch
```

Example output

```
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
granite   0/1     1            0           2s
granite   1/1     1            1           14s
```

1. Create a **Service** CR for the model inference. For example:

```
apiVersion: v1
kind: Service
metadata:
  name: granite
  namespace: rhaii-namespace
spec:
  selector:
    app: granite
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8000
```

2. Optional: Create a **Route** CR to enable public access to the model. For example:

```
apiVersion: route.openshift.io/v1
kind: Route
```

```
  metadata:
    name: granite
    namespace: rhaii-namespace
  spec:
    to:
      kind: Service
      name: granite
    port:
      targetPort: 80
```

3. Get the URL for the exposed route. Run the following command:

```
$ oc get route granite -n rhaii-namespace -o jsonpath='{.spec.host}'
```

Example output

```
granite-rhaii-namespace.apps.example.com
```

Verification

Ensure that the deployment is successful by querying the model. Run the following command:

```
curl -X POST http://granite-rhaii-namespace.apps.example.com/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "granite-3.1-2b-instruct-quantized.w8a8",
    "messages": [{"role": "user", "content": "What is AI?"}],
    "temperature": 0.1
  }'
```

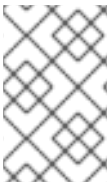
CHAPTER 6. DEPLOYING THE STANDALONE RED HAT AI INFERENCE CONTAINER ON IBM Z WITH IBM SPYRE ACCELERATORS

Deploy a language model on OpenShift Container Platform running on IBM Z with IBM Spyre AI accelerators by using the standalone Red Hat AI Inference container. You configure secrets, persistent storage, and a **Deployment** custom resource (CR) that pulls the model from Hugging Face and uses Red Hat AI Inference to inference serve the model.

For more information about installing the Spyre Operator, see the Spyre Operator for Z and LinuxONE User's Guide.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in as a user with **cluster-admin** privileges.
- Your cluster deployed on IBM Z has worker nodes with IBM Spyre AI accelerators installed.
- You have installed the IBM Spyre Operator in the cluster. For more information, see Installing the Spyre Operator.
- You have a Hugging Face account and have generated a Hugging Face access token.
- You have access to **registry.redhat.io** and the cluster can pull images from this registry.



NOTE

IBM Spyre AI accelerator cards support FP16 format model weights only. For compatible models, the Red Hat AI Inference inference engine automatically converts weights to FP16 at startup. No additional configuration is needed.

Procedure

1. Create the **Secret** custom resource (CR) for the Hugging Face token. The cluster uses the **Secret** CR to pull models from Hugging Face.
   a. Set the **HF_TOKEN** variable using the token you set in Hugging Face:

   ```
   $ HF_TOKEN=<your_huggingface_token>
   ```

   b. Set the cluster namespace to match where you deployed the Red Hat AI Inference image, for example:

   ```
   $ NAMESPACE=rhaii-namespace
   ```

   c. Create the **Secret** CR in the cluster:

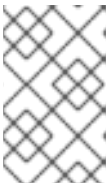
   ```
   $ oc create secret generic hf-secret --from-literal=HF_TOKEN=$HF_TOKEN -n
   $NAMESPACE
   ```

2. Create the Docker secret so that the cluster can download the Red Hat AI Inference image from the container registry. For example, to create a **Secret** CR that contains the contents of your local **~/.docker/config.json** file, run the following command:

```
$ oc create secret generic docker-secret --from-file=.dockercfg=$HOME/.docker/config.json --type=kubernetes.io/dockercfg -n rhaii-namespace
```

3. Create a **PersistentVolumeClaim** (**PVC**) custom resource (CR) and apply it in the cluster. The following example **PVC** CR uses a default IBM VPC Block persistence volume. You use the **PVC** as the location where you store the models that you download.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: model-cache
  namespace: rhaii-namespace
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 20Gi
  storageClassName: <STORAGE_CLASS_NAME>
```



NOTE

Configuring cluster storage to meet your requirements is outside the scope of this procedure. For more detailed information, see Configuring persistent storage.

4. Create a **Deployment** custom resource (CR) that pulls the model from Hugging Face and deploys the Red Hat AI Inference container. Reference the following example **Deployment** CR, which uses AI Inference to serve a Granite model with IBM Spyre AI accelerators.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: granite-spyre
  namespace: rhaii-namespace
  labels:
    app: granite-spyre
spec:
  replicas: 1
  selector:
    matchLabels:
      app: granite-spyre
  template:
    metadata:
      labels:
        app: granite-spyre
    spec:
      serviceAccountName: default
      volumes:
        - name: model-volume
```

```
        persistentVolumeClaim:
          claimName: model-cache
      - name: shm
        emptyDir:
          medium: Memory
          sizeLimit: "2Gi"
    initContainers:
      - name: fetch-model
        image: registry.redhat.io/ubi9/python-311:latest
        env:
          - name: HF_TOKEN
            valueFrom:
              secretKeyRef:
                name: hf-secret
                key: HF_TOKEN
          - name: HF_HOME
            value: /tmp/hf_home
          - name: HF_REPO_ID
            value: "ibm-granite/granite-3.3-8b-instruct"
        command:
          - /bin/bash
          - -lc
        args:
          - |
            set -euo pipefail
            mkdir -p /tmp/model
            if [ -z "$(ls -A /tmp/model 2>/dev/null)" ]; then
              echo "Installing huggingface_hub..."
              pip install --no-cache-dir -U huggingface_hub

              echo "Downloading model from Hugging Face: ${HF_REPO_ID}"
              echo "Using HF_HOME=${HF_HOME}"

              python -c 'import os; from huggingface_hub import snapshot_download;
snapshot_download(repo_id=os.environ["HF_REPO_ID"], local_dir="/tmp/model",
local_dir_use_symlinks=False, token=os.environ.get("HF_TOKEN"),
resume_download=True); print("Model download completed:", os.environ["HF_REPO_ID"])'
            else
              echo "Model already present in /tmp/model, skipping download."
            fi
        volumeMounts:
          - name: model-volume
            mountPath: /tmp/model
    containers:
      - name: vllm
        image: registry.redhat.io/{rhaii-registry-namespace}/vllm-spyre-rhel9:{rhaiis-version}
        command:
          - /bin/bash
          - -lc
          - |
            source /opt/rh/gcc-toolset-14/enable
            source /etc/profile.d/ibm-aiu-setup.sh
            exec python3 -m vllm.entrypoints.openai.api_server \
              --model=/tmp/model \
              --port=8000 \
              --served-model-name=spyre-model \
```

```
            --max-model-len=32768 \
            --max-num-seqs=32 \
            --tensor-parallel-size=4 \
            --enable-prefix-caching
        env:
          - name: HF_HOME
            value: /tmp/hf_home
          - name: FLEX_DEVICE
            value: VF
          - name: TOKENIZERS_PARALLELISM
            value: "false"
          - name: DTLOG_LEVEL
            value: error
          - name: TORCH_SENDNN_LOG
            value: CRITICAL
          - name: VLLM_SPYRE_USE_CB
            value: "1"
          - name: VLLM_SPYRE_REQUIRE_PRECOMPILED_DECODERS
            value: "1"
          - name: TORCH_SENDNN_CACHE_ENABLE
            value: "1"
          - name: VLLM_DT_CHUNK_LEN
            value: "512"
        ports:
          - name: http
            containerPort: 8000
        resources:
          requests:
            cpu: "16"
            memory: "160Gi"
            ibm.com/spyre_vf: "4"
          limits:
            cpu: "23"
            memory: "200Gi"
            ibm.com/spyre_vf: "4"
        volumeMounts:
          - name: model-volume
            mountPath: /tmp/model
            readOnly: true
          - name: shm
            mountPath: /dev/shm
```

Where:

namespace: rhaii-namespace

Specifies the deployment namespace. The value of **metadata.namespace** must match the namespace where you configured the Hugging Face **Secret** CR.

claimName: model-cache

Specifies the persistent volume claim name. The value of **spec.template.spec.volumes.persistentVolumeClaim.claimName** must match the name of the **PVC** that you created.

initContainers

Defines a container that runs before the main application container to download the required model from Hugging Face by using the **huggingface_hub** Python library. The

model download step is skipped if the model directory has already been populated, for example, from a previous deployment.

FLEX_DEVICE

Specifies the device type for IBM Spyre accelerators. Set to **VF** for virtual function mode.

TOKENIZERS_PARALLELISM

Disables tokenizer parallelism to prevent resource conflicts.

VLLM_SPYRE_USE_CB

Enables continuous batching for improved throughput on IBM Spyre accelerators.

VLLM_SPYRE_REQUIRE_PRECOMPILED_DECODERS

Requires precompiled decoders for optimal performance on Spyre accelerators.

TORCH_SENDNN_CACHE_ENABLE

Enables caching for the SendNN backend to improve model loading times.

ibm.com/spyre_vf

Requests IBM Spyre virtual function devices from the cluster. The number specifies how many Spyre AI accelerator devices to allocate.

mountPath: /dev/shm

Mounts the shared memory volume required for tensor parallel inference across multiple Spyre accelerators.

5. Increase the deployment replica count to the required number.

```
$ oc scale deployment granite-spyre -n rhaii-namespace --replicas=1
```

6. Optional: Watch the deployment and ensure that it succeeds, for example:

```
$ oc get deployment -n rhaii-namespace --watch
```

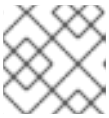
Example output:

```
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
granite-spyre   0/1     1            0           2s
granite-spyre   1/1     1            1           5m
```

7. Create a **Service** CR for the model inference. For example:

```
apiVersion: v1
kind: Service
metadata:
  name: granite-spyre
  namespace: rhaii-namespace
  labels:
    app: granite-spyre
spec:
  selector:
    app: granite-spyre
  ports:
    - name: http
      protocol: TCP
```

```
    port: 8000
    targetPort: 8000
  type: ClusterIP
```



NOTE

spec.selector.app must match the label in your **Deployment** pod.

8. Optional: Create a **Route** CR to enable public access to the model with TLS encryption. For example:

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: granite-spyre
  namespace: rhaii-namespace
  annotations:
    haproxy.router.openshift.io/timeout: 600s
spec:
  to:
    kind: Service
    name: granite-spyre
  port:
    targetPort: http
  tls:
    termination: edge
    insecureEdgeTerminationPolicy: Redirect
```

9. Get the URL for the exposed route. Run the following command:

```
$ oc get route granite-spyre -n rhaii-namespace -o jsonpath='{.spec.host}'
```

Example output:

```
granite-spyre-rhaii-namespace.apps.example.com
```

Verification

- Ensure that the deployment is successful by querying the model. Run the following command:

```
$ curl -X POST https://granite-spyre-rhaii-
namespace.apps.example.com/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "granite-3.3-8b-instruct",
    "messages": [{"role": "user", "content": "What is AI?"}],
    "temperature": 0.1
  }'
```

Example output:

```
{
  "id": "chatcmpl-abc123",
```

```
  "object": "chat.completion",
  "created": 1234567890,
  "model": "granite-3.3-8b-instruct",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "AI, or Artificial Intelligence, refers to the simulation of human intelligence in
machines..."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 50,
    "total_tokens": 62
  }
}
```


Additional resources

- Understanding deployments